

Amplifying Discrepancies: Exploiting Macro and Micro Inconsistencies for Image Manipulation Localization(Supplementary Material)

Anonymous submission

Appendix

Appendix A. IMDLBenco-based protocol

Recently, IMDLBenco has reorganized the landscape of image manipulation detection by retraining and evaluating existing methods under a unified standard, and has publicly released the weights of all models. We retrained our model in accordance with IMDLBenco’s CAT-protocol (Ma et al. 2024) data requirements, just like SparseViT (Su et al. 2025) and Mesorch (Zhu et al. 2025). In what follows, we detail the evaluation settings of this method.

IMDLBenco-based Setting: During training, we employ a balanced sampling strategy, treating CASIAv2 (Dong, Wang, and Tan 2013), COCO-Tamp (Kwon et al. 2022) (which includes three subsets), JpegRAISE (Dang-Nguyen et al. 2015), Fantastic Reality (Kniaz, Knyaz, and Remondino 2019), and IMD2020 (Novozamsky, Mahdian, and Saic 2020) as seven separate sub-datasets. In each epoch, we sample 1,840 images from each sub-dataset, resulting in a training batch of 12,880 images. During evaluation, a single fixed threshold of 0.5 is applied: pixels with predicted values above 0.5 are classified as manipulated, while those below 0.5 are classified as authentic. The final F1 and IoU scores are computed directly from these binary segmentation results.

Differences with original CAT-Net Protocol: (1) Unlike the official CAT-Net (Kwon et al. 2022), the IMDLBenco-based method omits all real images from both the CASIAv2 and RAISE datasets, resulting in a notably different composition of training data. (2) CAT-Net partitions a portion of its training set for validation before sampling, while the IMDLBenco workflow samples directly from the full training pool, leading to different train/validation compositions. (3) For the Fantastic Reality dataset, CAT-Net treats the real and fake subsets as two independent pools and samples separately from each for each epoch, whereas IMDLBenco combines both real and fake images into a single pool before stratified sampling. This results in a different number of samples drawn from this dataset.

Localization Performance: Table 1 presents a comprehensive comparison of localization performance across four standard datasets, with the highest scores in bold. Our method consistently achieves top performance, securing the best average F1 and IoU metrics (0.6999 and 0.6546, re-

spectively) among all evaluated models. Specifically, our approach outperforms all baselines on Coverage, NIST16, and CASIAv1 for both F1 and IoU, while remaining highly competitive on Columbia, where it ranks second only to SparseViT. Our method delivers balanced and robust performance across all benchmarks. These consistent gains demonstrate the strong generalization and effective manipulation localization of our model without the need for dataset-specific tuning.

Appendix B. Qualitative Visualization

As shown in Figure 1, we provide Grad-CAM visualizations to interpret the behaviors of different components. DPM captures fine-grained local inconsistencies by employing differential progressive operations, highlighting subtle tampering artifacts. In contrast, ICM focuses on global semantic discrepancies via iterative clustering. Notably, ICM exhibits stronger activation in authentic background regions, suggesting its ability to model macro-level contextual consistency and distinguish semantic-level mismatches. The fused result (DPM+ICM) effectively combines macro- and micro-level cues, leading to accurate and robust localization. These observations demonstrate the complementary nature of DPM and ICM.

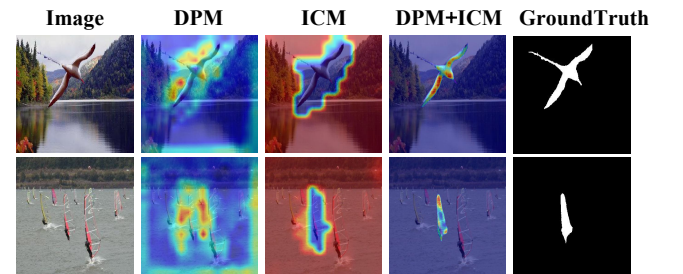


Figure 1: Qualitative results with Grad-CAM visualizations.

Appendix C. ICM Hyperparameter Analysis

The ICM module constitutes the core of our method. Extensive experiments show that an iteration count of $T = 3$ and 2 cluster centers deliver the best performance. We therefore

Model	Coverage		Columbia		NIST16		CASIAv1		Avg.	
	F1	IoU	F1	IoU	F1	IoU	F1	IoU	F1	IoU
MVSS-Net	0.4860	0.3886	0.7399	0.6721	0.3363	0.2593	0.5832	0.4907	0.5364	0.4527
CAT-Net	0.4273	0.3875	0.9150	0.8953	0.2521	0.2125	0.8081	0.7481	0.6006	0.5609
PSCC-Net	0.4475	0.3009	0.8841	0.8138	0.3457	0.2971	0.6304	0.4999	0.5769	0.4779
TruFor	0.4573	0.4149	0.8845	0.8593	0.3480	0.2964	0.8176	0.7746	0.6269	0.5863
SparseViT	0.5130	0.4720	0.9590	0.9380	0.3840	0.3310	0.8270	0.7750	0.6710	0.6290
Mesorch	0.5862	0.5360	0.8903	0.8766	0.3921	0.3367	0.8398	0.7980	0.6771	0.6356
FRD-Net (ours)	0.5953	0.5366	0.9267	0.9154	0.4349	0.3736	0.8427	0.7931	0.6999	0.6546

Table 1: Comparison of model performance using F1 and IoU metrics under IMDLBenCo-based protocol

Method	Columbia	COVER	CASIAV1	COCOGLIDE	NIST16	Avg.
init center randn	0.971	0.564	0.736	0.572	0.378	0.644
FRD-Net	0.973	0.601	0.829	0.626	0.412	0.688

Table 2: Comparison of random initialization and our model on F1 performance.

present the following ablation studies to analyze these design choices in detail.

Cluster Center Initialization : To assess the impact of cluster center initialization, we replace our average pooling initialization with random initialization. As shown in Table 2, our method achieves consistent improvements across all datasets. Specifically, average F1 rises by 4.4% (from 0.644 to 0.688), with notable gains on CASIAV1 (+11.3%), COVER (+3.7%), COCOGLIDE (+5.4%), and NIST16 (+3.4%). These results indicate that average pooling initialization provides a stronger starting point for clustering, stabilizes optimization, and allows the model to better capture manipulation cues, resulting in improved detection accuracy.

Number of Cluster Centers : We fix the number of iterations to $T = 3$ and investigate the impact of varying the number of cluster centers $K \in \{2, 4, 6\}$. When using $K = 2$, our method achieves F1 scores of 0.8287, 0.6007, 0.6259, 0.9764, and 0.4119 on CASIA, COVER, CocoGlide, Columbia, and NIST16, respectively. The corresponding average F1 score across all datasets is 0.6887. In comparison, increasing the number of cluster centers to $K = 4$ and $K = 6$ leads to lower average F1 scores, at 0.6552 and 0.6666, respectively. Setting $K = 2$ thus brings relative improvements of +5.1% over $K = 4$ and +3.3% over $K = 6$. Notably, while the NIST16 dataset shows a slight preference for $K = 4$ (0.4160, representing a 1.0% gain compared to $K = 2$), this advantage is marginal compared to the consistent improvements observed on the other four datasets. These results suggest that increasing K beyond two tends to fragment the already limited manipulated regions, weakens discriminative cues, and introduces additional optimization noise. Therefore, we set $K = 2$ as the default configuration for all subsequent experiments.

Number of Iterations : We fix the number of cluster centers to $K = 2$ and assess the influence of varying the itera-

Center Number	CASIA	COVER	NIST16	CocoGlide	Columbia	Avg.
2 (ours)	0.829	0.601	0.412	0.626	0.976	0.688
4	0.774	0.519	0.4160	0.598	0.969	0.655
6	0.806	0.552	0.413	0.604	0.958	0.667

Table 3: Influence of the number of cluster centers (K) in ICM on F1 performance.

tion count $T \in \{1, 2, 3\}$. With $T = 3$, our method obtains F1 scores of 0.8287, 0.6007, 0.6259, 0.9764, and 0.4119 on CASIA, COVER, CocoGlide, Columbia, and NIST16, respectively, giving the highest average value of 0.6887. Reducing the iteration count to $T = 2$ and $T = 1$ lowers the average F1 to 0.6658 and 0.6423, corresponding to relative decreases of 2.3% and 4.6% compared with $T = 3$. The largest gains appear on COVER and CASIA, while NIST16 still benefits slightly from additional refinement. These findings indicate that three refinement steps are sufficient to converge, whereas fewer iterations leave residual localization errors and weaker boundaries. Therefore, we set $T = 3$ as the default configuration in all subsequent experiments.

Method	CASIA	COVER	NIST16	CocoGlide	Columbia	Avg.
iter 1	0.773	0.542	0.377	0.562	0.958	0.642
iter 2	0.804	0.548	0.407	0.616	0.954	0.665
iter 3 (ours)	0.828	0.601	0.412	0.626	0.976	0.688

Table 4: Impact of the iteration count T on F1 performance.

References

- Dang-Nguyen, D.-T.; Pasquini, C.; Conotter, V.; and Boato, G. 2015. Raise: A Raw Images Dataset for Digital Image Forensics. In *Proceedings of the 6th ACM multimedia systems conference*, 219–224.
- Dong, J.; Wang, W.; and Tan, T. 2013. CASIA Image Tampering Detection Evaluation Database. In *2013 IEEE China Summit and International Conference on Signal and Information Processing*, 422–426. IEEE.
- Kniaz, V. V.; Knyaz, V.; and Remondino, F. 2019. The Point Where Reality Meets Fantasy: Mixed Adversarial Generators for Image Splice Detection. *Advances in Neural Information Processing Systems*, 32: 215–226.

- Kwon, M.-J.; Nam, S.-H.; Yu, I.-J.; Lee, H.-K.; and Kim, C. 2022. Learning JPEG Compression Artifacts for Image Manipulation Detection and Localization. *International Journal of Computer Vision*, 130(8): 1875–1895.
- Ma, X.; Zhu, X.; Su, L.; Du, B.; Jiang, Z.; Tong, B.; Lei, Z.; Yang, X.; Pun, C.-M.; Lv, J.; and Zhou, J. 2024. IMDL-BenCo: A Comprehensive Benchmark and Codebase for Image Manipulation Detection & Localization. In *Advances in Neural Information Processing Systems*, volume 37, 134591–134613.
- Novozamsky, A.; Mahdian, B.; and Saic, S. 2020. IMD2020: A Large-Scale Annotated Dataset Tailored for Detecting Manipulated Images. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision Workshops*, 71–80.
- Su, L.; Ma, X.; Zhu, X.; Niu, C.; Lei, Z.; and Zhou, J.-Z. 2025. Can We Get Rid of Handcrafted Feature Extractors? SparseViT: Nonsemantics-Centered, Parameter-Efficient Image Manipulation Localization Through Spare-Coding Transformer. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, 7024–7032.
- Zhu, X.; Ma, X.; Su, L.; Jiang, Z.; Du, B.; Wang, X.; Lei, Z.; Feng, W.; Pun, C.-M.; and Zhou, J.-Z. 2025. Mesoscopic Insights: Orchestrating Multi-Scale & Hybrid Architecture for Image Manipulation Localization. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, 11022–11030.